P5

Comprobaciones anti-debug y empaquetado

Parte I · RA5 · 2 h

Universidad de Almería · Edición de 27 de septiembre de 2026

Documento: Guion de práctica para el alumnado
Asignatura: Ingeniería Inversa y Análisis Forense Digital
Uso: Laboratorio docente en entorno controlado y sobre las muestras indicadas
Alumno/a: _____ Grupo: _____

Índice

1. Mapa de trabajo **2**

2. Material y preparación **2**

3. Trabajo en sesión (120 minutos) **2**

4. Entregable **2**

5. Evaluación **3**

6. Teoría necesaria para realizar la práctica **3**

7. Software requerido **3**

8. Guía detallada de realización **3**
8.1. 1. Identificar y ejecutar el control . . . 3
8.2. 2. Observar bajo GDB . . . 3
8.3. 3. Intervenir y controlar . . . 4
8.4. 4. Empaquetar una copia . . . 4
8.5. 5. Comparar estructura y comportamiento . . . 4

9. Tabla de resultados **4**

10. Errores frecuentes **4**

11. Actualización UPX-01 disponible — 28 de septiembre de 2026 **5**
11.1. Recepción y requisitos . . . 5
11.2. Comprobar, recuperar y volver a empaquetar . . . 5
11.3. Contraste estructural y de comportamiento . . . 5

Estado actual: UPX-01 1.0 validado. La actualización del 28 de septiembre de 2026, al final del documento, incorpora la muestra empaquetada y sus resultados medidos. Sustituye el estado UPX pendiente de la ficha histórica, que se conserva. El bloque anti-debug mantiene sus controles propios.

1 Mapa de trabajo



Figura 1: Secuencia de trabajo y puntos de control de la práctica.

El diagrama resume el orden recomendado. Cada transición debe quedar respaldada por una anotación, salida o fichero verificable en la entrega.

2 h · RA5 · 5 % de la nota de Parte I. Guion completo en LaTeX. Rúbrica única en criterios de evaluación.

2 Material y preparación

`make -C muestras/build x86 antidebug`; UPX validado previamente por el docente. Todas las intervenciones se hacen sobre copias de las muestras propias.

3 Trabajo en sesión (120 minutos)

1. Observar `antidebug-dbg` sin GDB y bajo GDB; anotar restricciones del entorno (20 min).
2. Localizar la llamada, observar su resultado y realizar una intervención controlada que permita continuar; explicar `PTRACE_TRACEME` y falsos positivos (35 min).
3. Empaquetar una copia y obtener otra desempaquetada (35 min):

```
upx -9 -o empaquetado muestras/build/salida/ejemplos-x86-strip-opt
upx -t empaquetado
upx -d -o recuperado empaquetado
sha256sum muestras/build/salida/ejemplos-x86-strip-opt empaquetado recuperado
cmp muestras/build/salida/ejemplos-x86-strip-opt recuperado
```

1. Contrastar tamaño, cabeceras, legibilidad y comportamiento (20 min).
2. Redactar qué demuestra cada comparación y qué queda fuera (10 min).

Si UPX rechaza el formato/build, el docente proporciona una variante propia validada; no se improvisa una descarga ajena durante la sesión. Una gráfica de entropía es apoyo opcional, no un requisito de una versión específica de Binwalk.

4 Entregable

Controles con/sin depurador/intervención; comparación original/empaquetado/recuperado; logs y hashes; explicación de límites. `upx -d` es automático, no desempaquetado manual.

5 Evaluación

Se aplica la rúbrica común: corrección técnica 40 %, evidencia reproducible 25 %, interpretación y límites 25 %, claridad 10 %. Todos los criterios se aplican al objetivo concreto de esta práctica; las ampliaciones no son necesarias para la máxima nota.

6 Teoría necesaria para realizar la práctica

`ptrace(PTRACE_TRACEME,...)` solicita que el proceso sea trazado por su padre. Si ya está bajo GDB suele fallar, pero también puede fallar por política, sandbox o permisos. El retorno es una señal ambiental, no un detector infalible. El programa decide qué hacer mediante instrucciones normales que pueden observarse o intervenirse.

Empaquetar transforma la representación y añade un *stub* que recupera código/datos. Secciones, entry point, tamaño, cadenas y entropía pueden cambiar. Entropía alta es compatible con compresión, cifrado y otros datos; no demuestra malware.

Los hashes identifican representaciones. Original, empaquetado y recuperado son objetos distintos; solo una comparación ejecutada determina si dos son idénticos.

7 Software requerido

GCC/Make, GDB, `readelf`, `file`, `sha256sum`, `cmp` y UPX validado.

```
bash PreparaciónEntorno/entorno/provision.sh --check
make -C muestras/build x86 antidebug
upx --version | head -n 2
mkdir -p resultados/P5
```

Si UPX no existe o rechaza el build, el docente proporciona una variante validada; no se descarga un ejecutable arbitrario durante la sesión.

8 Guía detallada de realización

8.1 1. Identificar y ejecutar el control

```
sha256sum muestras/build/salida/antidebug-dbg \
  muestras/build/salida/ejemplos-x86-strip-opt \
  > resultados/P5/muestras.sha256
set +e
muestras/build/salida/antidebug-dbg > resultados/P5/sin-gdb.out 2>&1
printf 'exit=%s\n' "$?" > resultados/P5/sin-gdb.exit
set -e
```

Registre kernel y política si el resultado difiere. No modifique la política del host.

8.2 2. Observar bajo GDB

```
gdb -q muestras/build/salida/antidebug-dbg
```

```
set pagination off
break ptrace
run
info registers rdi rsi rdx rcx
finish
info registers rax
```

```
disassemble /m main
```

Confirme el primer argumento según las cabeceras del entorno y observe el retorno. Localice la comparación y el salto. Distinga salida de `ptrace` y decisión del programa.

8.3 3. Intervenir y controlar

Tras registrar el retorno, fuerce temporalmente el valor esperado por la rama no detectada. Anote punto, valor previo/posterior y efecto. Ejecute después otro proceso sin GDB para comprobar que el fichero no cambió. Esto demuestra una decisión modificable, no que la técnica no exista.

8.4 4. Empaquetar una copia

```
cp muestras/build/salida/ejemplos-x86-strip-opt resultados/P5/original
upx -9 -o resultados/P5/empaquetado resultados/P5/original
upx -t resultados/P5/empaquetado > resultados/P5/upx-test.txt
upx -d -o resultados/P5/recuperado resultados/P5/empaquetado
sha256sum resultados/P5/original resultados/P5/empaquetado \
  resultados/P5/recuperado > resultados/P5/hashes.txt
cmp -s resultados/P5/original resultados/P5/recuperado
printf 'cmp_exit=%s\n' "$?" > resultados/P5/cmp.txt
```

`cmp_exit=0` significa igualdad byte a byte; otro valor se investiga conservando versiones y mensajes. No se declara igualdad porque ambos se ejecuten.

8.5 5. Comparar estructura y comportamiento

```
file resultados/P5/original resultados/P5/empaquetado resultados/P5/recuperado \
  > resultados/P5/file.txt
readelf -h -S resultados/P5/original > resultados/P5/original-elf.txt
readelf -h -S resultados/P5/empaquetado > resultados/P5/empaquetado-elf.txt
```

Compare tamaño, entry point, secciones, cadenas y análisis. Ejecute original y empaquetado con las mismas entradas dentro de la VM; compare stdout, stderr, retorno y efectos. Equivalencia para esos casos no prueba equivalencia total.

9 Tabla de resultados

Objeto/sesión	Hash/tamaño	Observación	Interpretación	Límite
anti-debug sin GDB	completar	completar	retorno/entorno	una técnica
anti-debug con GDB	completar	completar	efecto del tracer	falsos positivos
intervención	completar	completar	decisión forzada	no entrada válida
original/empaquetado/recuperado	completar	completar	representaciones	versión/opciones

10 Errores frecuentes

- Llamar malware a cualquier fichero empaquetado.
- Afirmar que `ptrace` siempre detecta GDB.
- Cambiar el original en vez de una copia.

- Confundir `upx -d` con desempaquetado manual.
- Interpretar `cmp` sin registrar su código.
- Esperar hashes iguales para objetos distintos.

11 Actualización UPX-01 disponible — 28 de septiembre de 2026

El bloque de empaquetado ya cuenta con una muestra benigna validada. Esta sección concreta los materiales y sustituye la preparación UPX pendiente de la ficha anterior, conservada íntegramente. El bloque anti-debug sigue usando antidebug-dbg y GDB con sus propios controles; no se sustituye por el ejercicio UPX.

11.1 Recepción y requisitos

Use el paquete del alumnado UPX-01. Incluye original y empaquetado, instrucciones, manifiesto, hashes y avisos del stub. No incluye la herramienta UPX ni el recuperado de control. El docente prepara UPX 4.2.2 antes de clase; la distribución ensayada es upx-ucl 4.2.2-3 de Ubuntu noble. Las instrucciones de preparación documentan procedencia y dependencias. Necesita Linux x86-64, glibc compatible, readelf, file, strings, sha256sum, cmp y stat. No necesita recompilar para realizar este bloque.

Después de extraer el paquete en una carpeta nueva, desde alumnado:

```
sha256sum -c SHA256SUMS
P5_UPX=upx
"$P5_UPX" --version
mkdir sesion-p5
```

Si el ejecutable se llama upx-ucl, cambie P5_UPX a ese nombre o a la ruta que indique el docente. No descargue otra versión durante la práctica para ocultar un fallo.

11.2 Comprobar, recuperar y volver a empaquetar

```
"$P5_UPX" -t empaquetado > sesion-p5/test.log 2>&1
printf '%s\n' "$?" > sesion-p5/test.exit
"$P5_UPX" -d -o sesion-p5/recuperado empaquetado \
  > sesion-p5/recuperar.log 2>&1
printf '%s\n' "$?" > sesion-p5/recuperar.exit
cmp -s original sesion-p5/recuperado
printf '%s\n' "$?" > sesion-p5/cmp.exit
sha256sum original empaquetado sesion-p5/recuperado > sesion-p5/hashes.txt
stat -c '%n %s bytes' original empaquetado sesion-p5/recuperado \
  > sesion-p5/tamanos.txt
"$P5_UPX" -9 -o sesion-p5/propio original > sesion-p5/empaquetar.log 2>&1
printf '%s\n' "$?" > sesion-p5/empaquetar.exit
```

Lea los retornos antes de continuar: un fallo de recuperación no produce un resultado válido. cmp=0 indica igualdad, 1 diferencia y 2 error. Conserve cualquier diferencia entre una columna de progreso de UPX y el tamaño final medido con stat; no copie una cifra de progreso como tamaño final sin comprobarla. Nunca reutilice la carpeta.

11.3 Contraste estructural y de comportamiento

Guarde file, readelf -h -lW -SW y strings -a para las tres representaciones. Complete una tabla con tamaño, entry ELF, número de program headers y section headers. Busque una cadena del original también en el empaquetado: no suponga que todas desaparecen. La herramienta puede

describir la envoltura UPX de modo distinto a la aplicación recuperada; explique qué objeto está analizando en cada paso.

Dentro de la VM, ejecute al menos estas nueve pruebas con resultados separados:

```
for P5_nombre in original empaquetado recuperado; do
  P5_bin="./$P5_nombre"
  if [ "$P5_nombre" = recuperado ]; then P5_bin=./sesion-p5/recuperado; fi
  for P5_caso in valida incorrecta vacia; do
    case "$P5_caso" in
      valida) P5_entrada=R3v3rs1ng_UAL ;;
      incorrecta) P5_entrada=AAAAAAAAAAAA ;;
      vacia) P5_entrada='' ;;
    esac
    "$P5_bin" "$P5_entrada" \
      > "sesion-p5/$P5_nombre-$P5_caso.out" \
      2> "sesion-p5/$P5_nombre-$P5_caso.err"
    printf '%s\n' "$?" > "sesion-p5/$P5_nombre-$P5_caso.exit"
  done
done
sha256sum -c SHA256SUMS
```

Repita sin argumentos. El mensaje de uso incorpora argv[0], por lo que invocar rutas diferentes cambia ese texto; documente esa diferencia en lugar de atribuirla automáticamente al empaquetado. Distinga esa observación de los casos con argumento.

```
original -- UPX -9 --> empaquetado -- UPX -d --> recuperado
   |                       |                        |
   +------- mismas entradas y registro -------------+
   +----------- cmp y SHA-256 ----------------------+
```

Entregue la tabla, logs UPX/cmp, hashes, controles de comportamiento y explicación de qué prueba cada comparación. Conserve los entregables anti-debug del otro bloque. No equipare -t con benignidad ni -d con desempaquetado manual.